

4. PROJECT PLANNING PHASE

4.1 Work Breakdown Structure and Schedule

The project can be divided into work packages: requirement gathering, data-model design, interface prototyping, database setup, player and team management, match entry, calculation functions, dashboard development, reporting, testing, documentation and demonstration preparation. Each package should have a clear deliverable and completion condition.

A sample schedule may allocate the first week to requirements and scope, the second to design and interface planning, weeks three and four to database and core workflows, week five to calculations and reports, week six to testing and correction, and the final period to documentation and presentation. The exact schedule should reflect the academic calendar and team size.

Dependencies should guide the order of work. Statistical definitions and data relationships need to be settled before report calculations are implemented. A small working prototype should be demonstrated early so users can comment on the workflow before substantial development effort is invested. Tasks can be tracked using a board with To Do, In Progress, Review and Done states.

Each task should have an owner, target date and acceptance condition. Short progress reviews help identify blockers such as missing sample scorecards, unclear dismissal rules or installation problems. The schedule should include time for integration, testing and document review rather than assigning all available time to feature coding.

4.2 Resource and Technology Planning

Required resources include development computers, a supported programming environment, source-control software, a database, sample scorecards, test users and documentation tools. Dependencies should be recorded in a requirements file or equivalent setup document. A virtual environment helps isolate project libraries and makes installation more reproducible.

Team roles may include coordinator, developer, database designer, tester and documentation owner. In a small academic group, one person may perform several roles, but responsibilities should remain explicit. A coach, scorer or experienced cricket player can review terminology and calculation rules. This domain review is especially useful for overs notation, extras, dismissals and format-specific differences.

Sample data should cover typical and unusual situations: a dismissed batter, a not-out innings, zero dismissals, a maiden over, extras, a player with no bowling record and a match with incomplete data. Synthetic data is suitable for testing when real records are unavailable. If real player information is used, permission and privacy should be considered.

Technology choices should be based on project size, available skills, maintainability and deployment needs. Open-source tools reduce licensing cost, but they still require version management and documentation. A clean-environment installation test should be planned before the final submission to ensure that the application is not dependent on one developer's computer.

4.3 Risk Assessment and Mitigation

A risk register should record the risk description, likelihood, impact, owner and mitigation. Data-entry errors are a major risk because incorrect source values lead to incorrect statistics. Mitigations include validation, clear field labels, review against scorecards and test cases using known values. Formula errors can be reduced by documenting definitions and checking results independently.

Scope expansion can threaten completion. Features should be classified as essential, desirable or future work, and major additions should be reviewed before implementation. Technical risks include incompatible library versions, database corruption and deployment problems. These can be reduced through dependency pinning, version control, backups and clean-environment testing.

Usability risk can be addressed through early demonstrations and task-based feedback. Privacy risk can be reduced by collecting only necessary information, restricting access and limiting exported fields. The team should also consider the risk of misleading interpretation when reports use small samples or incomplete records.

A contingency plan should identify features that can be deferred if time becomes limited. The minimum viable product should preserve the core value of the project: reliable player records, match entry, correct calculations and basic reports. Risks should be reviewed throughout development and updated when new information becomes available.

4.4 Quality Plan and Change Management

Quality planning defines how requirements, design, code, data and documentation will be reviewed. Each requirement should map to test cases. Calculation logic should be isolated into reusable functions and covered by unit tests. Database relationships should be checked with valid and invalid examples. User-interface workflows should be reviewed for clarity and consistency.

Version control provides a history of changes and helps team members coordinate work. Commits should describe meaningful changes, and major updates should be reviewed before integration. After changes to formulas, database structure or shared interface components, the team should run relevant tests and regression checks.

A change log can record feature additions, bug fixes and known limitations. Changes to statistical definitions require particular care because they can alter historical reports. The team should document whether existing records need recalculation or migration. Data backups should be taken before changes that affect stored information.

Documentation quality is included in the quality plan. Screenshots should show the final application, diagrams should match the implemented architecture and the user guide should use the same field names as the interface. Time should be reserved for proofreading, checking references and rehearsing the demonstration.